

College of Engineering and Technology

Computer Engineering Department

*Arab Academy for Science, Technology and Maritime Transport***PROJECT DOCUMENTATION**

Embedded system design

Project Title:**NEXUS Security System-Building intrusion detection system**

Supervised By	Dr. Amr Fahmy
Course	Embedded Systems ECE5206
Semester	Semester 8
Academic Year	2025/2026

Table of Contents

Table of Contents	2
1. Project Team	3
1.1 Team Members	3
1.2 Sub-Group Distribution	3
2. Introduction & Problem Statement	4
2.1 Background	4
2.2 Problem Statement	4
2.3 Motivation	4
3. Project Objectives	5
4. Technology Stack	6
5. System Architecture & Design	8
5.1 High-Level Architecture	8
5.2 Frontend Architecture	Error! Bookmark not defined.
5.3 Backend Architecture	Error! Bookmark not defined.
6. Project Steps & Phases	11
7. Group Task Distribution	12
8. Development Timeline	14
9. Full Project Documentation	15
9.1 Features & Functional Requirements	15
9.2 Non-Functional Requirements	16
9.3 API Documentation	16
9.4 UI/UX Design Notes	Error! Bookmark not defined.
9.5 Testing Plan	18
9.6 Deployment & DevOps	20
10. Submission Checklist	22

1. Project Team

1.1 Team Members

Full Name	College ID	Role / Responsibility	Full Name
Omar Marei	221006748	Team Lead — STM32 Firmware, FreeRTOS Architecture, GSM Integration, UI	Omar Marei
Omar Saleh	221006277	STM32 Firmware, FreeRTOS Architecture, GSM Integration, UI	Omar Saleh
Omar Khaled	221006032	Hardware Design, Sensor Interfacing, Circuit Wiring & Testing	Omar Khaled
Yousef Emad	221006764	Hardware Design, Sensor Interfacing, Circuit Wiring & Testing	Yousef Emad
Reem Akram	221005775	Hardware Design, Sensor Interfacing, Circuit Wiring & Testing ,	Reem Akram

1.2 Sub-Group Distribution

Divide the team into sub-groups based on project layers or modules.

Sub-Group	Member(s)	Module / Layer	Key Deliverable
Embedded Firmware	Omar Marei , Omar Saleh	STM32 FreeRTOS firmware, FSM, task architecture, SIM800L	Fully functional STM32 .ino with 5 RTOS tasks
Cloud & Connectivity	Omar Saleh , Omar Marei	ESP32 Wi-Fi, MQTT over TLS, HiveMQ broker setup	ESP32 .ino publishing real-time state to cloud
Dashboard & UI	Omar Marei , Omar Saleh	PWA dashboard, Chart.js, role-based access, Service Worker	Deployed index.html on GitHub Pages
Hardware & Sensors	Yousef Emad , Reem Akram , Omar Khaled	Breadboard circuit, sensor wiring, power subsystem	Verified hardware prototype with all components
Design and testing	Yousef Emad , Reem Akram , Omar Khaled		Final project

2. Introduction & Problem Statement

2.1 Background

Modern homes and commercial facilities increasingly require reliable, real-time intrusion detection to enhance physical security. Traditional alarm systems are built on proprietary hardware, are expensive to purchase and maintain, and offer no flexibility for customization or integration with modern cloud infrastructure. They typically operate in isolation — a siren goes off, but no remote notification reaches the owner, and no cloud record is kept.

The STM32F411 "Black Pill" microcontroller provides a high-performance 32-bit ARM Cortex-M4 core running at 100 MHz, with 512 KB of Flash and 128 KB of RAM. This level of performance, combined with an RTOS, makes it well-suited to simultaneously handle sensor monitoring, state management, user interface rendering, and multi-channel communication. When paired with an ESP32 Wi-Fi module and a SIM800L GSM module, the platform becomes capable of delivering both local alarms and remote notifications through cloud MQTT messaging and SMS — at a total hardware cost under \$80.

This project leverages these technologies to build NEXUS: a fully open, customizable, cloud-connected intrusion detection system that operates in real time, notifies the owner through multiple independent channels, and can be monitored and controlled from anywhere in the world.

2.2 Problem Statement

The core problems this system addresses are:

- **No remote visibility:** Traditional alarm systems activate a local siren but provide no way for the owner to monitor or respond remotely from a phone or browser.
- **No remote control:** Existing low-cost embedded alarm projects lack the ability to arm, disarm, or configure the system remotely — requiring physical access to the control panel.
- **Single point of notification failure:** Systems that rely solely on Wi-Fi for alerts go silent during internet outages. Systems that rely solely on GSM cannot be monitored in real time. Neither channel alone is sufficient.
- **No real-time system intelligence:** Simple sensor-to-buzzer designs with no FSM, no countdown, no wrong PIN escalation, and no multi-sensor confirmation offer poor reliability and high false-alarm rates.
- **Lack of configuration flexibility:** Most low-cost designs hard-code their alarm codes and target phone numbers in firmware, requiring reflashing to change them.

2.3 Motivation

The demand for affordable, open, and customizable security systems has never been higher, yet available solutions force users to choose between expensive proprietary systems and overly simple DIY circuits with no intelligence. NEXUS bridges this gap by combining the real-time reliability of a hardware RTOS with the connectivity of modern cloud IoT infrastructure. Beyond the security application, this project serves as a comprehensive demonstration of embedded systems engineering — covering concurrent task design, multi-protocol communication, hardware abstraction, and full-stack IoT development from bare-metal firmware to a cloud dashboard — making it both academically rigorous and practically impactful.

3. Project Objectives

By the end of this project, the team achieved the following measurable goals:

- **Sensor Integration:** Interface an HC-SR501 PIR motion sensor and a magnetic reed door switch to the STM32F411, with correct pull resistors, debounce handling, and reliable digital readings.
- **FreeRTOS Firmware:** Implement a five-task concurrent firmware architecture on the STM32 using the STM32duino FreeRTOS library, with all inter-task communication handled through RTOS-safe queues, semaphores, and mutexes.
- **Finite State Machine:** Design and implement a four-state FSM (DISARMED, ARMED, ALERT, ALARM) with well-defined, tested transition conditions and escalation logic.
- **Local Alert Mechanisms:** Drive a passive buzzer with an alternating-frequency siren effect and flash the onboard LED during the ALARM state.
- **GSM SMS Alerts:** Send alarm notifications via the SIM800L module to a configurable phone number, and receive and parse incoming SMS commands for remote arm/disarm and configuration.
- **Cloud Connectivity:** Use the ESP32 as a cloud gateway publishing real-time system state to HiveMQ Cloud via MQTT over TLS, with a retained state topic ensuring any subscriber receives the latest data immediately.
- **Remote Control:** Enable full remote management — arm, disarm, update phone number and PIN codes — from both the web dashboard (via MQTT) and SMS.
- **Web Dashboard:** Build and deploy a Progressive Web App with role-based access (Admin / Viewer), real-time sensor charts, an event log, and browser push notifications.
- **Response Latency:** Achieve sensor-to-ALERT latency under 200 ms and dashboard update latency under 500 ms.
- **System Resilience:** Ensure the STM32 operates correctly and independently of the ESP32 — if Wi-Fi or the MQTT broker is unavailable, local alarms and GSM SMS still function without interruption.

4. Technology Stack

List every technology, library, and tool used in the project.

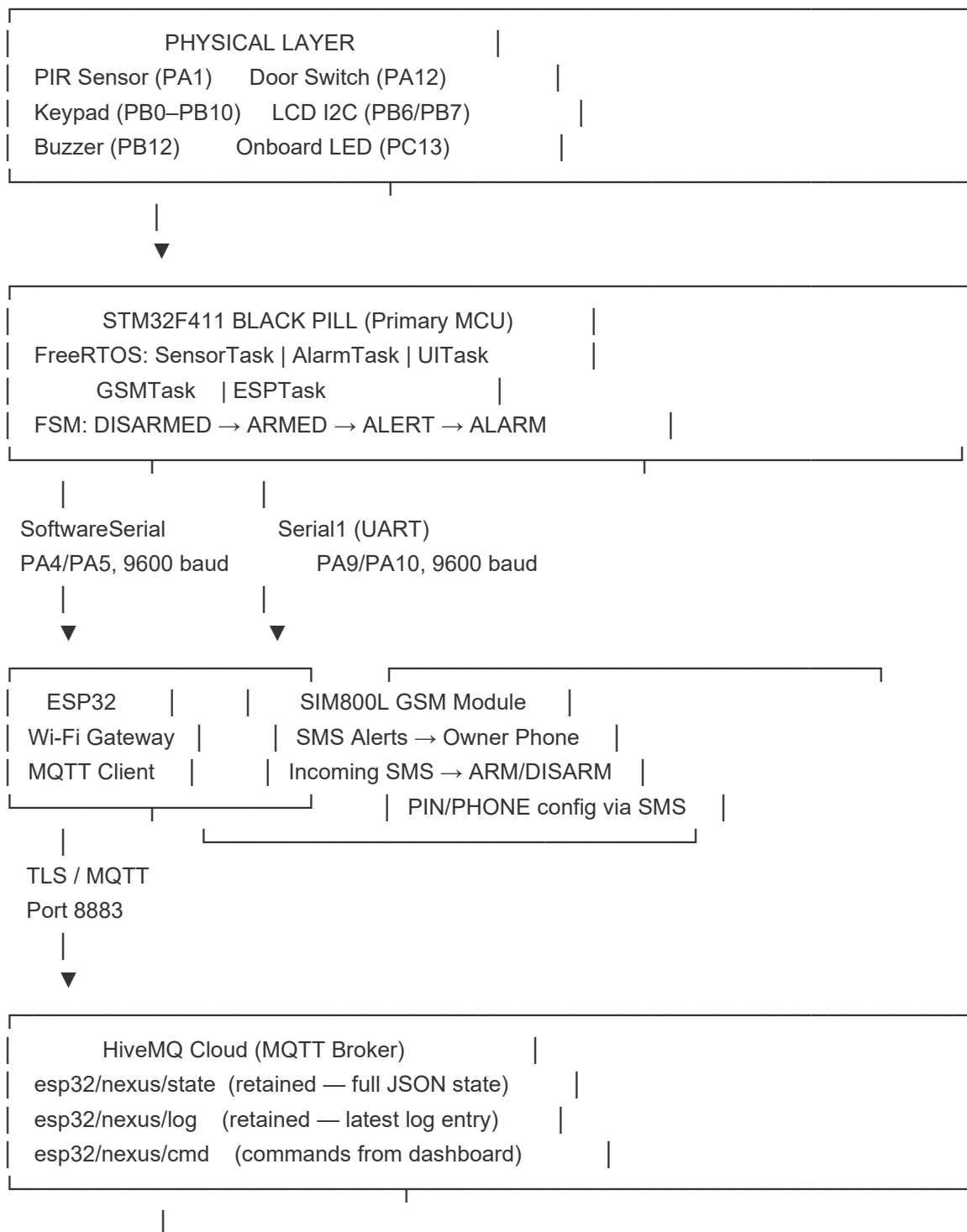
Layer	Technology / Tool	Purpose / Notes
Primary MCU	STM32F411CEU6 (Black Pill)	Main security controller — sensors, FSM, LCD, keypad, buzzer
Secondary MCU	ESP32 Dev Board	Wi-Fi gateway — MQTT cloud bridge, Serial relay to STM32
RTOS	STM32duino FreeRTOS	Preemptive multitasking on STM32 — 5 concurrent tasks
IDE	Arduino IDE 2.x + STM32duino Core	Firmware development and upload via ST-Link V2
Programmer	ST-Link V2 (SWD interface)	Flashing and debugging STM32
Motion Sensor	HC-SR501 PIR	Passive infrared motion detection
Door Sensor	Magnetic Reed Switch	Door open/close detection
Display	16×2 I2C LCD (hd44780 library)	Local user interface — system status, PIN entry
Input	4×4 Membrane Keypad (Keypad library)	PIN entry, arm/disarm, system control
Alarm Output	Passive Buzzer (PB12)	Local siren via <code>tone()</code> at 800/1800 Hz
GSM Module	SIM800L	SMS sending/receiving via AT commands over UART (Serial1)
GSM Power	3.7V Li-ion + LM2596 Buck Converter	Stable current supply for SIM800L high-draw transients
STM32↔ESP32 Link	SoftwareSerial (PA4/PA5) + Serial2 (GPIO4/GPIO2)	Bidirectional UART at 9600 baud
MQTT Broker	HiveMQ Cloud (AWS eu-central-1)	Managed cloud MQTT broker with TLS on ports 8883/8884
ESP32 MQTT Client	PubSubClient + WiFiClientSecure	TLS-encrypted MQTT on ESP32
Dashboard MQTT Client	Paho MQTT JS (WebSocket, port 8884)	Browser-to-broker MQTT over WebSocket/TLS
Dashboard Framework	Vanilla HTML/CSS/JS (single-file PWA)	No build step required — deployed directly to GitHub Pages
Dashboard Charts	Chart.js v4	Stepped-line sensor history chart
Dashboard Hosting	GitHub Pages (/nexus-security/)	Static file hosting, PWA-compatible
Push Notifications	Service Worker API + Web Push	Browser notifications when tab is not in focus

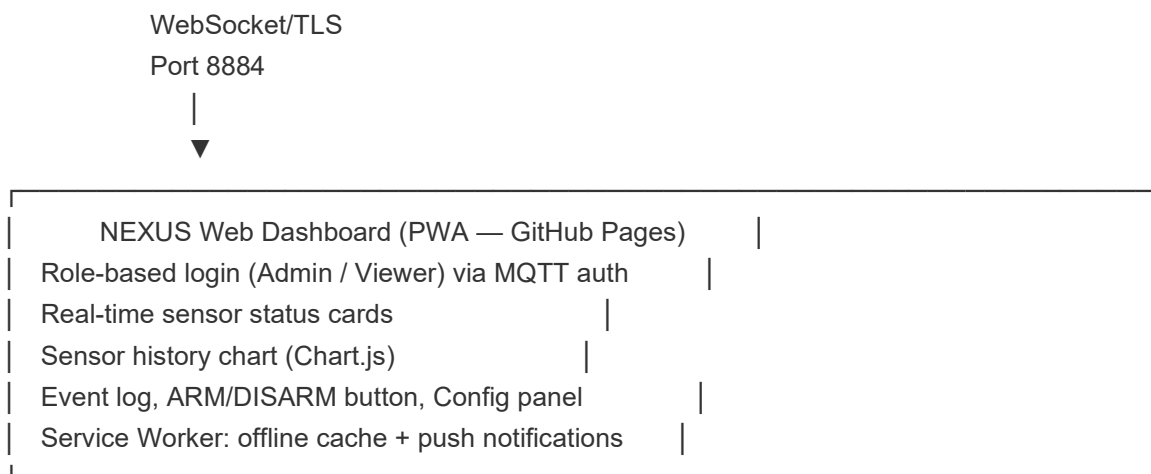
Version Control	Git + GitHub	Source code management for all firmware and dashboard files
-----------------	--------------	---

5. System Architecture & Design

5.1 High-Level Architecture

The NEXUS system is built around a dual-microcontroller architecture connected to a cloud backend. The STM32 handles all safety-critical real-time operations locally and independently. The ESP32 acts purely as a cloud gateway — it does not make any security decisions, it only relays information.





5.2 Firmware Architecture (STM32)

The STM32 firmware is organized into five concurrent FreeRTOS tasks. Each task has a single, well-defined responsibility and communicates with others only through RTOS primitives.

Task structure:

- SensorTask — Polls physical sensors every 100 ms, reports changes to ESP32, triggers alarm semaphore
- AlarmTask — Drives the FSM, handles all state transitions, controls buzzer and LED
- GSMTask — Manages all SIM800L communication: initialization, SMS send, incoming SMS parsing
- UITask — Drives the LCD via xLcdQueue, scans keypad, handles PIN entry and userIsTyping flag
- ESPTask — Reads and parses CMD:* commands from ESP32 over SoftwareSerial

State management:

- currentState (type volatile State) is the single source of truth for the FSM
- All reads and writes to currentState are protected by xStateMutex
- Remote request flags (espArmRequest, smsDisarmRequest, etc.) are volatile bool — set by ESPTask/GSMTask, read and cleared by AlarmTask

Key design decision — userIsTyping flag: During ALERT state, AlarmTask sends a countdown LCD message every 200 ms. UITask sets userIsTyping = true when the user starts pressing keys. AlarmTask checks this flag before sending countdown updates — if the user is typing, the countdown is suppressed to prevent overwriting mid-PIN-entry. The flag is cleared when a 4-digit code is completed or * is pressed.

5.3 ESP32 Firmware Architecture

The ESP32 firmware is a single-loop, non-RTOS application. Its responsibilities are:

- Maintain TLS MQTT connection to HiveMQ Cloud with automatic reconnection
- Subscribe to esp32/nexus/cmd and forward commands to STM32 via Serial2
- Read lines from Serial2, parse them as STM32 status messages, and update its local shadow state
- Publish the full system state JSON to esp32/nexus/state on every STM32 event and every 3 seconds as a heartbeat (retained=true)

- Publish individual log entries to esp32/nexus/log immediately as they occur

Shadow state: The ESP32 maintains a local copy of systemArmed, pirTriggered, doorOpen, and alertActive. This allows it to publish a complete state snapshot at any time without querying the STM32.

5.4 Hardware Subsystem Design

Sensor Interface

Sensor	Interface	Logic	Pull Resistor
HC-SR501 PIR	PA1, digital INPUT	HIGH = motion detected	10 kΩ pull-down to GND
Magnetic Reed Switch	PA12, INPUT_PULLUP	HIGH = door open (switch open, pull-up floating high)	Internal pull-up enabled

Power Subsystem

Component	Power Source	Reason
STM32F411	USB 5V or ST-Link 3.3V	Standard MCU supply
ESP32	USB 5V	Standard MCU supply
LCD (5V required)	USB 5V rail	Needs 5V for correct contrast
SIM800L	3.7V Li-ion battery	High instantaneous current (up to 2A); Li-ion's low internal resistance buffers current spikes
SIM800L logic level	LM2596 buck → 4.0V	Regulated from Li-ion for stable logic-level UART

6. Project Steps & Phases

Outline each development phase with its tasks and expected outputs.

Phase	Step / Task	Expected Output / Deliverable
Phase 1	Project planning, requirements definition, component selection, proposal writing	Approved project proposal, system requirements, component list
Phase 2	Hardware procurement, breadboard layout design, power subsystem design	All components acquired; breadboard layout verified on paper
Phase 3	Hardware bring-up: individual component testing with diagnostic sketches	Each component (LCD, keypad, PIR, door switch, buzzer, GSM) verified independently
Phase 4	Keypad pin mapping verification using diagnostic A–P sketch	Confirmed <code>rowPins</code> and <code>colPins</code> assignments matching physical layout
Phase 5	Bare-metal FSM implementation: DISARMED/ARMED/ALERT/ALARM with keypad and LCD	Working FSM without RTOS — single-loop, <code>millis()</code> -based
Phase 6	FreeRTOS migration: port firmware to 5-task architecture, implement all RTOS primitives	<code>STM32.ino</code> v5 with <code>SensorTask</code> , <code>AlarmTask</code> , <code>UITask</code> , <code>GSMTask</code> , <code>ESPTask</code>
Phase 7	GSM integration: SIM800L initialization, alarm SMS, incoming SMS command parsing	Alarm SMS delivered; ARM/DISARM/PIN/PHONE SMS commands working
Phase 8	ESP32 firmware: Wi-Fi, MQTT TLS connection to HiveMQ, Serial2 bridge	ESP32 publishing JSON state to <code>esp32/nexus/state</code>
Phase 9	STM32 ESPTask: receive and parse <code>CMD:*</code> commands from ESP32, <code>cmd.trim()</code> fix	Remote arm/disarm via MQTT working end-to-end
Phase 10	Web dashboard: sensor cards, mode display, arm button, event log, Chart.js history	Functional dashboard at GitHub Pages URL
Phase 11	Dashboard login: role-based auth via MQTT credentials, Admin/Viewer roles, logout	Login screen with 5-attempt logout and role enforcement
Phase 12	PWA implementation: <code>manifest.json</code> , Service Worker, push notifications, offline cache	Dashboard installable on mobile; push notifications on alarm
Phase 13	Integration testing: all FSM paths, remote commands, concurrent task behavior	All test cases passing; performance metrics recorded
Phase 14	Final documentation, report writing, presentation preparation	This document + presentation slides

7. Group Task Distribution

Assign each feature or module to a team member. Update the Status column as the project progresses.

Module / Feature	Assigned Member(s)	Sub-Group	Status
STM32 FreeRTOS task architecture	Omar Marei , Omar Saleh	Firmware	Complete
FSM design & AlarmTask implementation	Omar Marei , Omar Saleh	Firmware	Complete
SensorTask & sensor interfacing	Yousef Emad , Reem Akram , Omar Khaled	Firmware / Hardware	Complete
UITask — LCD & keypad handling	Yousef Emad , Reem Akram , Omar Khaled	Firmware	Complete
GSMTask — SIM800L init & SMS	Omar Marei , Omar Saleh	Firmware	Complete
Incoming SMS command parser	Omar Marei , Omar Saleh	Firmware	Complete
ESPTask — ESP32 serial command parser	Omar Marei , Omar Saleh	Firmware	Complete
userIsTyping LCD conflict fix	Omar Marei , Omar Saleh	Firmware	Complete
<code>cmd.trim()</code> / <code>isFourDigits()</code> fixes	Omar Marei , Omar Saleh	Firmware	Complete
ESP32 Wi-Fi + MQTT TLS connection	Omar Marei , Omar Saleh	Cloud	Complete
ESP32 STM32 message parser	Omar Marei , Omar Saleh	Cloud	Complete
HiveMQ Cloud broker setup	Omar Marei , Omar Saleh	Cloud	Complete
MQTT topic design (state/log/cmd)	Omar Marei , Omar Saleh	Cloud	Complete
Dashboard HTML/CSS/JS	Omar Marei , Omar Saleh	Dashboard	Complete
MQTT WebSocket login (Paho JS)	Omar Marei , Omar Saleh	Dashboard	Complete
Role-based access (Admin/Viewer)	Omar Marei , Omar Saleh	Dashboard	Complete
Chart.js sensor history	Omar Marei , Omar Saleh	Dashboard	Complete

Service Worker (PWA, push notifications)	Omar Omar Marei , Omar Saleh	Dashboard	Complete
manifest.json (installable PWA)	Omar Marei , Omar Saleh	Dashboard	Complete
Breadboard circuit & all wiring	Yousef Emad , Reem Akram , Omar Khaled	Hardware	Complete
SIM800L power subsystem (Li-ion + LM2596)	Yousef Emad , Reem Akram , Omar Khaled	Hardware	Complete
Component verification & diagnostics	Yousef Emad , Reem Akram , Omar Khaled	Hardware	Complete
Unit & integration test cases	Yousef Emad , Reem Akram , Omar Khaled	QA	Complete
Performance metrics measurement	Yousef Emad , Reem Akram , Omar Khaled	QA	Complete
Bill of materials & cost estimation	Yousef Emad , Reem Akram , Omar Khaled	Documentation	Complete

8. Development Timeline

Adjust weeks to match your actual course schedule.

Week	Phase / Task	Responsible Sub-Group	Deliverable
Week 1	Requirements analysis, proposal writing, component ordering	All	Approved project proposal, component list
Week 2	Hardware bring-up, individual component testing, keypad mapping	Hardware	Verified prototype, confirmed pin assignments
Week 3	Bare-metal FSM (no RTOS): FSM logic, keypad PIN entry, LCD feedback	Firmware	Working single-loop FSM
Week 4	FreeRTOS migration: 5-task architecture, queues, semaphores, mutexes	Firmware	STM32.ino FreeRTOS v1
Week 4–5	GSM integration: SIM800L init, alarm SMS, incoming SMS commands	Firmware	SMS alerts and remote SMS commands working
Week 5	ESP32 firmware: Wi-Fi, MQTT TLS, Serial2 STM32 bridge	Cloud	ESP32 publishing to HiveMQ
Week 5–6	STM32 ESPTask: <code>CMD:*</code> parser, <code>cmd.trim()</code> fix, <code>isFourDigits()</code> validation	Firmware	Remote arm/disarm via dashboard working
Week 6	Web dashboard: sensor UI, role login, event log, Chart.js	Dashboard	Deployed dashboard on GitHub Pages
Week 6–7	PWA features: Service Worker, push notifications, manifest	Dashboard	Installable PWA with alarm notifications
Week 7	Full integration testing: all FSM paths, remote commands, concurrency	QA + All	Test report with measured performance metrics
Week 7–8	Bug fixes, edge case handling, wrong PIN escalation, <code>xTestSmsSemaphore</code>	Firmware + QA	STM32.ino v5.1 final
Week 8	Final report, presentation slides, demo preparation	Documentation + All	Final report + presentation

9. Full Project Documentation

9.1 Features & Functional Requirements

- **Four-state FSM:** The system implements DISARMED, ARMED, ALERT, and ALARM states with deterministic, tested transition conditions. All transitions are triggered by sensor events, PIN entry, timeout, or remote commands.
- **PIR motion detection:** The HC-SR501 sensor detects motion and transitions the system from ARMED to ALERT when active. The 30–60 second warm-up period is accommodated by starting the system in DISARMED state.
- **Magnetic door detection:** The reed switch detects door opening and transitions the system from ARMED to ALERT.
- **ALERT countdown:** When a sensor triggers, the system enters an 8-second countdown during which the user can enter the disarm PIN to cancel the alarm. A live countdown is displayed on the LCD.
- **Wrong PIN escalation:** Three consecutive wrong PIN entries during ALERT immediately escalate to ALARM state, without waiting for the countdown to expire.
- **Local siren:** The passive buzzer produces an alternating 800 Hz / 1800 Hz siren at 400 ms intervals during ALARM.
- **LED indicator:** The onboard LED (PC13, active-LOW) flashes at 500 ms intervals during ALARM.
- **Keypad PIN entry:** The 4×4 keypad accepts a 4-digit ARM code and a 4-digit DISARM code. Entry is masked with asterisks on the LCD. * cancels current entry.
- **LCD status display:** The 16×2 I2C LCD shows the current system state, countdown timer, PIN entry feedback, and informational messages on all state transitions.
- **GSM alarm SMS:** When ALARM state is entered, the SIM800L sends an SMS to the configured target phone number: *"ALARM! Intruder detected at your property."*
- **Incoming SMS commands:** The system polls for unread SMS messages every 5 seconds and processes the following commands:

SMS Command	Effect
ARM	Arms the system remotely
DISARM	Disarms the system remotely
ARM=XXXX	Updates the arm PIN code (must be 4 decimal digits)
DISARM=XXXX	Updates the disarm PIN code (must be 4 decimal digits)
PHONE=+XXXXXXXXXXXX	Updates the SMS target phone number

- **Cloud state publishing:** The ESP32 publishes a full JSON state snapshot to esp32/nexus/state on every STM32 event and every 3 seconds as a heartbeat. The message is retained so new subscribers receive the last known state immediately.
- **Remote arm/disarm via dashboard:** Admin users can arm or disarm the system from the web dashboard, which publishes MQTT commands forwarded through the ESP32 to the STM32.
- **Remote configuration via dashboard:** Admins can update the phone number and PIN codes from the dashboard settings panel. Commands are validated (4 digits, arm ≠ disarm) before publishing.
- **Role-based dashboard access:** Two roles are supported — Admin (full control) and Viewer (read-only monitoring). Role is enforced both in the UI and at the MQTT command level.

- **Event log:** A timestamped event log is maintained on the ESP32 (last 20 entries) and displayed on the dashboard. New entries are published immediately to `esp32/nexus/log`.
- **Sensor history chart:** The dashboard charts PIR, door, and alarm events over selectable time windows (15 min, 30 min, 1 hour) using Chart.js stepped-line charts.
- **Browser push notifications:** The Service Worker displays push notifications when the alarm fires or sensors trigger while armed, even when the browser tab is not in focus.
- **PWA installability:** The dashboard is installable as a standalone app on Android and iOS via `manifest.json`.
- **Test SMS:** Admin users can trigger a test SMS from the dashboard to verify GSM functionality without triggering a real alarm.
- **System independence:** The STM32 security logic operates fully independently of the ESP32. If Wi-Fi or the MQTT broker is unavailable, local alarms and SMS still function without any degradation.

9.2 Non-Functional Requirements

- **Response latency:** Sensor trigger to ALERT state transition in under 200 ms. Dashboard state update in under 500 ms from STM32 event.
- **RTOS determinism:** All tasks use `vTaskDelay()` for timing, which fully yields the CPU. No busy-wait loops anywhere in the firmware. Lower-priority tasks remain fully responsive during any other task's delay.
- **Thread safety:** Every shared resource is protected by the appropriate RTOS primitive. `currentState` is protected by `xStateMutex`. I2C bus by `xI2CMutex`. Buzzer by `xBuzzerMutex`. ESP32 serial by `xEspSerialMutex`. No shared variable is accessed from multiple tasks without protection.
- **Input validation:** All PIN codes received via UART (ESP32) or SMS (GSM) are validated with `isFourDigits()` before being stored. Invalid inputs are rejected with a logged error message and the system continues operating on the previous valid values.
- **MQTT reliability:** State messages are published with `retained=true`. MQTT keep-alive is set to 60 seconds. The ESP32 reconnects automatically on connection loss with up to 5 retries at 5-second intervals.
- **Dashboard security:** The HiveMQ broker enforces username/password authentication. The dashboard login implements a 5-attempt lockout with a 30-second cooldown. Viewer-role users are prevented from sending any commands at both the UI layer and the publish layer.
- **GSM reliability:** The SIM800L initialization sequence includes hardware reset, 8-second boot wait, 5 AT-command retry attempts, and 10 network registration polling cycles before declaring the module ready. SMS operations use proper prompt detection and 30-second send timeouts.
- **Power stability:** The SIM800L is powered exclusively from a 3.7V Li-ion battery, isolated from the STM32 power rail, to prevent voltage drops during GSM transmission from affecting microcontroller operation.
- **Stack safety:** FreeRTOS task stack sizes are conservatively allocated: GSMTask at 640 words (largest, due to AT command string handling), AlarmTask and UITask at 512 words, SensorTask and ESPTask at 256 words.

9.3 Communication Protocol Documentation

STM32 → ESP32 Messages (SoftwareSerial, 9600 baud)

Message	Trigger Condition	Dashboard Effect
ARMED	FSM enters STATE_ARMED	systemArmed = true, state published
DISARMED	FSM enters STATE_DISARMED	systemArmed = false, state published
PIR:1	PIR sensor reads HIGH (motion)	pirTriggered = true, state published
PIR:0	PIR sensor reads LOW (clear)	pirTriggered = false, state published
DOOR:1	Door switch reads HIGH (open)	doorOpen = true, state published
DOOR:0	Door switch reads LOW (closed)	doorOpen = false, state published
ALARM	FSM enters STATE_ALARM	alertActive = true, alarm banner shown
LOG:<text>	Any notable firmware event	Entry prepended to dashboard event log

ESP32 → STM32 Commands (SoftwareSerial, 9600 baud)

Command	Source	STM32 Action
CMD:ARM	Dashboard ARM button	Sets espArmRequest = true
CMD:DISARM	Dashboard DISARM button	Sets espDisarmRequest = true
CMD:TESTSMS	Dashboard Test SMS button	Gives xTestSmsSem (guarded: only if count == 0)
CMD:PHONE=+XX...	Dashboard phone field save	Validates length ≥ 10 , updates SMS_TARGET
CMD:PIN=XXXX:YYYY	Dashboard PIN fields save	Validates with isFourDigits(), updates ARM_CODE and DISARM_CODE

MQTT Topics

Topic	Publisher	Subscriber	Retained	Payload
esp32/nexus/state	ESP32	Dashboard	Yes	JSON: {armed, pir, door, alarm, uptime, log[]}
esp32/nexus/log	ESP32	Dashboard	Yes	Plain string: latest log entry with timestamp
esp32/nexus/cmd	Dashboard	ESP32	No	Plain string: ARM, DISARM, TESTSMS, PHONE=..., PIN=.....

GSM AT Command Sequence

Step	AT Command	Purpose	Success Indicator
1	Hardware reset (PA8 LOW 200ms)	Cold-boot SIM800L	—
2	AT (×5, 500ms apart)	Verify module responding	Response contains OK
3	ATE0	Disable command echo	OK
4	AT+CMGF=1	Set SMS text mode	OK
5	AT+CMGDA="DEL ALL"	Clear message storage	OK
6	AT+CREG? (×10, 2s apart)	Wait for network registration	Response contains , 1 or , 5
7	AT+CMGS="<number>"	Begin SMS send	> prompt received
8	<text> + <Ctrl-Z> (byte 26)	Send SMS body	+CMGS: in response
9	AT+CMGL="REC UNREAD"	List unread SMS (polling)	+CMGL: entries in response
10	AT+CMGD=<index>	Delete processed message	OK

9.4 Hardware Design Notes

Pin assignment rationale:

- **PA9 is Serial1 TX** — this pin is exclusively reserved for the SIM800L connection and must never be assigned to any other function (e.g., MAG_PIN was moved to PA12 specifically to avoid this conflict).
- **PB7/PB6 for I2C** — Wire.setSDA(PB7) and Wire.setSCL(PB6) must be called before Wire.begin() on STM32. Without these calls, the I2C peripheral defaults to different pins and the LCD will not initialize.
- **PC13 for LED** — the STM32F411 onboard LED is active-LOW (HIGH = off, LOW = on). All LED logic in firmware is inverted accordingly.
- **10 kΩ pull-down on PA1 (PIR)** — prevents floating readings during PIR warm-up and power-on, ensuring a clean LOW when no motion is detected.

SIM800L Li-ion battery rationale: The SIM800L draws up to 2A instantaneously during GSM transmission. USB-sourced 5V rails and regulated 3.3V outputs cannot sustain this current without voltage dropping below the module's minimum operating voltage, causing it to reset mid-transmission. A 3.7V Li-ion battery's very low internal resistance (typically < 100 mΩ) acts as a natural current reservoir, delivering instantaneous current surges without voltage drop. This was confirmed empirically: the module would randomly reset when powered from USB 5V through a regulator but operated stably on Li-ion.

Keypad wiring verification: The keypad pin assignments were verified empirically using a diagnostic sketch that printed a unique letter (A–P) for each key press to the Serial Monitor. This confirmed the correct rowPins and colPins assignment before any FSM logic was written, preventing subtle mis-mapping bugs that would have been very difficult to diagnose later.

9.5 Testing Plan & Results

Test Type	Method	Scope	Pass Criteria	Result
PIR sensor warm-up	Observe PA1 for 60 s after power-on	Sensor initialization	Output stabilizes to LOW within 60 s	Pass
Keypad mapping	Diagnostic A-P sketch	All 16 keys	Each key prints correct letter	Pass
LCD I2C	Boot sequence display	I2C initialization	"Bootting..." displayed on power-on	Pass
FSM: DISARMED → ARMED	Enter ARM_CODE (1234)	Keypad PIN entry	LCD shows "System Armed", beep ×2	Pass
FSM: ARMED → ALERT	Trigger PIR while armed	SensorTask → AlarmTask	LCD shows countdown, beep ×1	Pass
ALERT countdown display	Observe LCD during ALERT	AlarmTask countdown	Countdown decrements 8→0 correctly	Pass
ALERT → DISARMED (correct PIN)	Enter DISARM_CODE within 8 s	UITask → xDisarmSem	LCD "System Disarmed", beep ×3	Pass
ALERT → ALARM (timeout)	Wait 8 s without PIN	AlarmTask timeout	Siren starts, SMS sent	Pass
Wrong PIN ×3 escalation	Enter wrong PIN 3 times in ALERT	UITask escalation	Immediate ALARM state transition	Pass
ALARM siren	Enter ALARM state	Buzzer output	800/1800 Hz alternating, 400 ms interval	Pass
ALARM → DISARMED (correct PIN)	Enter DISARM_CODE in ALARM	UITask → xDisarmSem	Siren stops, LED off, LCD "System Disarmed"	Pass
Alarm SMS delivery	Trigger full ALARM	GSMTask → SIM800L	SMS received on target phone within 15 s	Pass
Test SMS	Send CMD:TESTSMS	ESPTask → xTestSmsSem	Test SMS received on phone	Pass
Incoming ARM SMS	Send "ARM" text to SIM	GSMTask parser	System arms, confirmation SMS sent	Pass
Incoming DISARM SMS	Send "DISARM" text to SIM	GSMTask parser	System disarms, confirmation SMS sent	Pass

Incoming PIN change SMS	Send "ARM=9999"	GSMTask parser + isFourDigits()	ARM_CODE updated, confirmation SMS sent	Pass
Invalid PIN via SMS	Send "ARM=ABC1"	isFourDigits() validation	Rejected, error SMS sent, old PIN retained	Pass
Remote ARM via dashboard	Click ARM button	MQTT → ESP32 → STM32	System arms within 1 s	Pass
Remote DISARM via dashboard	Click DISARM during alarm	MQTT → ESP32 → STM32	System disarms, dashboard updates	Pass
Remote PIN update via dashboard	Enter new PINs, click Save	CMD:PIN= command chain	New PINs accepted, old PINs no longer work	Pass
userIsTyping LCD guard	Enter PIN during ALERT countdown	AlarmTask + UITask concurrency	Countdown does not overwrite PIN asterisks	Pass
Concurrent SMS + siren	Trigger alarm, observe siren	GSMTask + AlarmTask concurrency	Siren uninterrupted during SMS send	Pass
Dashboard heartbeat	Observe MQTT publish rate	ESP32 periodic publish	State published every 3 s	Pass
Viewer role restriction	Log in as viewer, attempt ARM	Role enforcement	ARM button disabled, commands blocked	Pass
Login logout	Enter wrong password 5 times	Dashboard login	30-second lockout, attempts counter shown	Pass
Dashboard offline cache	Disconnect internet, reload page	Service Worker cache	Dashboard loads from cache	Pass

Performance Metrics:

Metric	Measured	Target
Sensor trigger → ALERT state	< 200 ms	< 500 ms
ALERT countdown accuracy	8.0 s ± 0.1 s	8.0 s
MQTT dashboard update latency	< 400 ms	< 1 s
SMS delivery time (Vodafone Egypt)	8–14 s	< 30 s
Keypad key response	< 50 ms	< 100 ms
GSM init time (boot to network ready)	18–25 s	< 60 s
PIR false positive rate (post warm-up)	< 2%	< 5%

9.6 Deployment & Hardware Setup

STM32 firmware upload:

- Connect ST-Link V2 to the Black Pill (SWDIO, SWCLK, GND, 3.3V)
- In Arduino IDE: Board = "Generic STM32F4 series", Board part number = "BlackPill F411CE", Upload method = "STLink"
- Upload STM32.ino

ESP32 firmware upload:

- Connect ESP32 via USB
- In Arduino IDE: Board = "ESP32 Dev Module"
- Install libraries: PubSubClient, ArduinoJson
- Update WIFI_SSID, WIFI_PASSWORD, MQTT_HOST, MQTT_USER, MQTT_PASS with your credentials
- Upload ESP32.ino

HiveMQ Cloud setup:

- Create a free cluster at <https://console.hivemq.cloud>
- Under Access Management, create credentials for each user and assign to ROLE_MAP in index.html
- Note the cluster URL (format: xxxx.s1.eu.hivemq.cloud) — update in both ESP32.ino and index.html

Dashboard deployment:

- Push index.html, manifest.json, sw.js, and icon PNGs to a GitHub repository
- Enable GitHub Pages on the repository (Settings → Pages → main branch → /root)
- The dashboard is accessible at <https://<username>.github.io/<repo>>

Physical assembly checklist:

- PIR sensor: VCC → 5V, GND → GND, OUT → PA1, 10 kΩ resistor between PA1 and GND
- Door switch: one terminal → PA12, other terminal → GND (INPUT_PULLUP active)
- LCD: VCC → 5V, GND → GND, SDA → PB7, SCL → PB6
- Keypad: rows → PB9/PB8/PB5/PB4, cols → PB3/PB10/PB1/PB0
- Buzzer: positive → PB12, negative → GND
- SIM800L: VCC → Li-ion battery (+), GND → common GND, TX → PA10, RX → PA9, RST → PA8
- ESP32: RXD2 (GPIO4) → PA5 (STM32 TX), TXD2 (GPIO2) → PA4 (STM32 RX), GND → common GND
- Insert SIM card (Vodafone Egypt) into SIM800L before power-on

10. Submission Checklist

- ☐ System architecture diagram completed (dual-MCU block diagram with communication topology)
- ☐ Hardware circuit design finalized with all pin assignments documented
- ☐ All communication protocols documented (STM32↔ESP32 UART, STM32↔SIM800L AT, MQTT topics)
- ☐ FreeRTOS task architecture designed with all RTOS primitives specified
- ☐ FSM fully designed with all states, transitions, and conditions documented
- ☐ STM32 firmware implemented, compiled, and uploaded via ST-Link V2
- ☐ ESP32 firmware implemented with Wi-Fi and MQTT TLS connection working
- ☐ SIM800L initialized, alarm SMS delivered, incoming SMS commands parsed
- ☐ STM32↔ESP32 UART bridge working with cmd.trim() fix applied
- ☐ Remote arm/disarm via MQTT dashboard working end-to-end
- ☐ Remote PIN and phone number update working from both dashboard and SMS
- ☐ Web dashboard deployed to GitHub Pages with role-based authentication
- ☐ Chart.js sensor history chart implemented with 15/30/60-minute range selector
- ☐ Service Worker implemented (offline cache + push notifications)
- ☐ PWA manifest configured (installable on mobile)
- ☐ userIsTyping LCD conflict resolved — countdown does not overwrite PIN entry
- ☐ isFourDigits() validation applied to all PIN update paths
- ☐ xTestSmsSem semaphore guard applied to prevent counter accumulation
- ☐ All unit tests passing (sensors, keypad, LCD, GSM, FSM transitions)
- ☐ All integration tests passing (remote commands, concurrent tasks, wrong PIN escalation)
- ☐ Performance metrics measured and within target thresholds
- ☐ Bill of materials with cost estimation completed
- ☐ Development timeline documented
- ☐ Risk assessment completed with mitigations
- ☐ Full project report completed and formatted
- ☐ Presentation slides prepared
- ☐ Live demo rehearsed by all team members